Enums and Pattern matching

Enums allow you to define a type by enumerating its possible values.
First, we'll define and use an enum to show how an enum can encode meaning along with data.

Then we'll look at how pattern matching in the match expression makes it easy to run different code for different values of an enum.

Defining an Enum

Say we need to work with IP addresses. Currently, two major standards are used for IP addresses: version four and version six.
These are the only possibilities for an IP address that our program will come across: we can enumerate all possible values, which is where enumeration gets its name.

We can express this concept in code by defining an IpAddrKind enumeration and listing the possible kinds an IP address can be, V4 and V6 as follows:

```
enum IpAddrKind {
    V4,
    V6,
}
```

IpAddrKind is now a custom data type that we can use elsewhere in our code.

For example we can define a function that takes any 'IpAddrKind' as parameter:

```
fn route(ip_type: IpAddrKind) { }
```

And call it as follows:

```
route(IpAddrKind::V4);
route(IpAddrKind::V6);
```

By doing so we are now only defining the kind of the IP address but not the address data;
We might then use structs to achive that:

```
enum IpAddrKind {
    V4,
    V6,
}

struct IpAddr{
    kind: IpAddrKind,
    address: String,
}

let home = {
    kind: IpAddrKind::V4,
    address: String::From("127.0.0.1"),
};

let loopback = {
```

```
    kind: IpAddrKind::V6,
    addresss: String::From("::1"),
};
```

We can represent the same concept in a more concise way using just an enum, rather than an enum inside a struct, by putting data directly into each enum variant.
This new definition of the 'IpAddr' enum says that both V4 and V6 variants will have associated 'String' values:

```
enum IpAddr {
    V4(String),
    V6(String),
}

let home = IpAddr::V4(String::from("127.0.0.1"));

let loopback = IpAddr::V6(String::from("::1"));
```

or also

```
enum IpAddr {
    V4(u8, u8, u8, u8),
    V6(String),
}

let home = IpAddr::V4(127, 0, 0, 1);

let loopback = IpAddr::V6(String::from("::1"));
```

Here is an example of enum that includes different types embedded in its variants:

```
enum Message {
    Quit,
    Move { x: i32, y: i32 },
    Write(String),
    ChangeColor(i32, i32, i32),
}
```

they are:

- 'Quit' has no data associated with it at all;
- 'Move' includes an anonymous struct inside it;
- 'Write' includes a single String;
- 'ChangeColor' includes three i32 values.

We could have used structs to achive the same result, but we couldn't as easily define a function to take any of these kinds of messages as we could with the 'Message' enum defined before which is a single type;

Even more,just as we're able to define methods on structs using 'impl', we're also able to define methods on enums, here is an example:

```
impl Message {
    fn call(&self) {
        // method body would be defined here
    }
}
```

```
let m = Message::Write(String::from("hello"));
m.call();
```

The Option Enum and its advantages over Null Values

The Option type is used in many places because it encodes the very common scenario in which a value could be something or it could be nothing.
Expressing this concept in terms of the type system means the compiler can check whether you've handled all the cases you should be handling;

Rust doesn't have the null feature that many other languages have. Null is a value that means there is no value there. In languages with null, variables can always be in one of two states: null or not-null.

As such, Rust does not have nulls, but it does have an enum that can encode the concept of a value being present or absent. This enum is `Option<T>`, and it is defined by the standard library as follows:

```
enum Option<T> {
    Some(T),
    None,
}
```

In addition, so are its variants: you can use 'Some' and 'None' directly without the 'Option::' prefix.

The `Option<T>` enum is still just a regular enum, and 'Some(T)' and None are still variants of type `Option<T>`.
The `<T>` syntax is a feature of Rust we haven't talked about yet, it's a generic type parameter which means the 'Some' variant of the 'Option' enum can hold one piece of data of any type.
Here is an example:

```
enum Option<T> {
    Some(T),
    None,
}

let some_number = Some(5);
let some_string = Some("a string");

let absent_number: Option<i32> = None;
```

If we use 'None' rather than 'Some', we need to tell Rust what type of `Option<T>` we have, because the compiler can't infer the type that the 'Some' variant will hold by looking only at a 'None' value.\

When we have a 'Some' value, we know that a value is present and the value is held within the 'Some'.

When we have a 'None' value, in some sense, it means the same thing as null: we don't have a valid value.

Why explicit typing for None? Because Rust is statically typed, every variable must commit to one specific type. Even an empty 'None' ne eds a clear label (like `Option<i32>`) defining exactly what type of data it is authorized to hold if it were a 'Some'.

The real advantage over standard Null: Unlike traditional null pointers, you cannot accidentally use an `Option<T>` directly as a 'T'. The compiler physically prevents runtime crashes by forcing you to explicitly handle the 'None' scenario (e.g., via 'match') before it allows you to extract and use the inner value.

We also need to remember that you have to convert an `Option<T>` to a 'T' before you can perform 'T' operations with it, for instance this code won't compile because they are physically two distinct types:

```
let x: i8 = 5;
let y: Option<i8> = Some(5);

let sum = x + y;
```

In general, in order to use an `Option<T>` value, you want to have code that will handle each variant.
You want some code that will run only when you have a 'Some(T)' value, and this code is allowed to use the inner 'T'.

You want some other code to run if you have a 'None' value, and that code doesn't have a 'T' value available. The 'match' expression is a control flow construct that does just this when used with enums.

The 'match' Control Flow operator

'match' allows you to compare a value against a series of patterns and then execute code based on which pattern matches.
Patterns can be made up of literal values, variable names, wildcards, and many other things;

Let's use coins as an example of using 'match';
We can write a function that can take an unknown United States coin and, in a similar way as the counting machine, determine which coin it is and return its value in cents:

```
enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter,
}

fn value_in_cents(coin: Coin) -> u32 {
    match coin {
        Coin::Penny => 1,
        Coin::Nickel => ,5
        Coin::Dime => 10,
        Coin::Quarter => 25,
    }
}
```

When the match expression executes, it compares the resulting value against the pattern of each arm, in order.
If a pattern matches the value, the code associated with that pattern is executed.

If that pattern doesn't match the value, execution continues to the next arm, much as in a coin-sorting machine.

The code associated with each arm is an expression, and the resulting value of the expression in the matching arm is the value that gets returned for the entire match expression.
We can use curly brackets to allows us to run multiple lines of code:

```
fn value_in_cents(coin: Coin) -> u32 {
    match coin {
        Coin::Penny => {
            println!("Lucky penny!");
            1
        },
        Coin::Nickel => 5,
        Coin::Dime => 10,
        Coin::Quarter => 25,
    }
```

```
}
```

Patterns that bind to values

Another useful feature of match arms is that they can bind to the parts of the values that match the pattern. This is how we can extract values out of enum variants.

For example, a 'Coin' enum in which the 'Quarter' variant also holds a 'UsState' value:

```
enum UsState {
    Alabama,
    Alaska,
    // --snip--
}

enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter(UsState),
}
```

In the match expression for this code, we add a variable called state to the pattern that matches values of the variant 'Coin::Quarter'. When a 'Coin::Quarter' matches, the state variable will bind to the value of that quarter's state, continuing with the previous assumptions:

```
fn value_in_cents(coin: Coin) -> u32 {
    match coin {
        Coin::Penny => 1,
        Coin::Nickel => 5,
        Coin::Dime => 10,
        Coin::Quarter(state) => {
            println!("State quarter from {:?}!", state);
            25
        },
    }
}
```

If we were to call 'value_in_cents(Coin::Quarter(UsState::Alaska))', coin would be 'Coin::Quarter(UsState::Alaska)'. By doing so we also get the inner value of the enum.

Matching with `Option<T>`

Let's say we want to write a function that takes an `Option<i32>` and, if there's a value inside, adds 1 to that value. If there isn't a value inside, the function should return the None value and not attempt to perform any operations.

```
fn plus_one(x: Option<i32>) -> Option<i32> {
    match x {
        None => None,
        Some(i) => Some(i+1),
    }
}

let five = Some(5);
let six = plus_one(five);
let none = plus_one(None);
```

Matches are exhaustive

If we didn't handle the 'None' case, this code will cause a bug but luckily, it's a bug Rust knows how to catch:

```
fn plus_one(x: Option<i32>) -> Option<i32> {
    match x {
        Some(i) => Some(i + 1),
    }
}
```

Matches in Rust are exhaustive: we must exhaust every last possibility in order for the code to be valid.
Especially in the case of `Option<T>`, when Rust prevents us from forgetting to explicitly handle the 'None' case, it protects us from assuming that we have a value when we might have null.

The _ placeholder

Rust also has a pattern we can use when we don't want to list all possible values.

If we only care about the values 1, 3, 5, and 7, we don't want to have to list out 0, 2, 4, 6, 8, 9 all the way up to 255.
The _ pattern will match any value.

By putting it after our other arms, the _ will match all the possible cases that aren't specified before it.
The () is just the unit value, so nothing will happen in the _ case.

```
let some_u8_value = 0u8;
match some_u8_value {
    1 => println!("one"),
    3 => println!("three"),
    5 => println!("five"),
    7 => println!("seven"),
    _ => (),
}
```

Coincise control flow with `if let`

If we want to define just one case in our match expression that does something we would do:

```
let some_u8_value = Some(0u8);

match some_u8_value {
    Some(3) => println!("three"),
    _ => (),
}
```

But we can also express the same logic using `if let` in a more concise way:

```
let some_u8_value = Some(0u8);

if let Some(3) = some_u8_value {
    println!("three");
}
```

The syntax `if let` takes a pattern and an expression separated by an equal sign. It works the same way as a 'match', where the expression is given to the 'match' and the pattern is its first arm.
Using `if let` means less typing, less indentation, and less boilerplate code. However, you lose the exhaustive

checking that match enforces.

We can include an 'else' with an `if let`. The block of code that goes with the 'else' is the same as the block of code that would go with the _ case in the 'match' expression that is equivalent to the `if let` and 'else'.

Using the 'Coin' example from above, if we wanted to count every non-Quarter while also announcing the states of the quarters, we have this two equivalent possibilites:

```
let mut count = 0;

match coin {
    Coin::Quarter(state) => println!("State quarter from {:?}!", state),
    _ => count += 1,
}
```

or

```
let mut count = 0;

if let Coin::Quarter(state) = coin {
    println!("State quarter from {:?}!", state);
} else{
    count += 1;
}
```